

Testing and Verification Report - Rising Waters

1. Automated Integration Testing

We configured a python test script test_vercel_app.py utilizing the Flask test client to verify routes, parameters, and redirects without spinning up a live server.

Test Matrix Summary

All 9 test scenarios passed successfully in 0.057 seconds:

Test ID	Function tested	Input Parameters	Expected Outcome	Status
01	GET / (Home)	None	Status 200, contains "Rising Waters"	PASSED
02	GET /Predict (Form)	None	Status 200, contains "Predict" input	PASSED
03	POST /predict (Low-risk)	Cloud: 5%, Annual: 1200mm, June-Sep, 200mm	Status 200, contains "Low Risk" variables	PASSED
04	POST /predict (High-risk)	Cloud: 90%, Annual: 4500mm, June-Sep, 3200mm	Status 200, contains "High Risk" variables	PASSED
05	POST /predict (Negative)	Cloud: 50%, Annual: 2000mm, June-Sep, 1200mm	Status 200, displays "No Negative warning"	PASSED
06	POST /predict (Cloud)	Cloud: 105%, Annual: 2000mm, June-Sep, 1200mm	Status 200, displays "Cloud warning"	PASSED
07	POST /predict (Logical Sum)	Annual: 1000mm, Sea level Sum: 1200mm	Status 200, displays "Logical sum error"	PASSED
08	GET /chance (Warning)	Prob: 98.55%, inputs: Status 200	Status 200, displays warning and 98.55%	PASSED
09	GET /no_chance (Safe)	Prob: 92.10%, inputs: Status 200	Status 200, displays safe and 92.10%	PASSED

2. Model Performance Assessment

Evaluated on the 25% test split (random state 10):

- Accuracy Score: 96.55% (28 correct classifications out of 29 samples).
- F1-Score: 96.55%
- Precision / Recall: 96.55%

Confusion Matrix Outputs

- True Negatives (No Flood): 12
- False Negatives: 1
- True Positives (Flood Risk): 16
- False Positives: 0
- *Generalization*: The model displays 0% false positives, ensuring warning alerts are only triggered

under genuine hazard parameters.